

Python for Artificial Intelligence

Day 2: Conditionals & Reading Errors

Month 1: Python Mastery | LLM & Agentic AI Track

Today's Agenda

- Hour 1: Conditional logic, if / elif / else, boolean logic
- Hour 2: Nested conditionals, truthiness, and common conditional bugs
- 15-Minute Break
- Hour 3: Reading a traceback, syntax vs. runtime errors, finding the failing line

Day 1 Quick Review (10 mins)

Before we start, try these on your own:

1. Create a variable score and assign it the value 87.

2. Print: "Your score is X" using an f-string.

3. What data type does 87 belong to? What about 87.0?

4. Convert the string "42" into an integer and print its type.

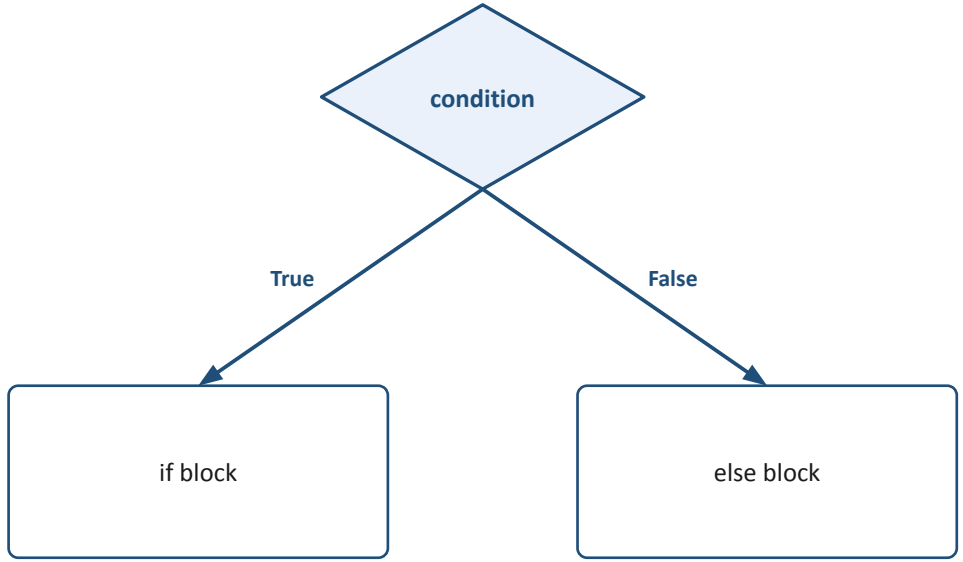
5. What does input() always return, before you convert it?



Hour 1: Conditional Logic

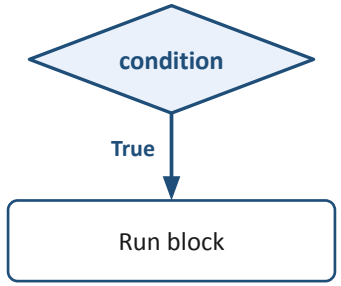
What Is Conditional Logic?

- Code that makes decisions based on True/False conditions
- Lets a program choose which block of code runs
- Built using if, elif, and else
- Real-life analogy: "If it's raining, take an umbrella. Else, wear sunglasses."



if Statement

Runs a block only when the condition is True.

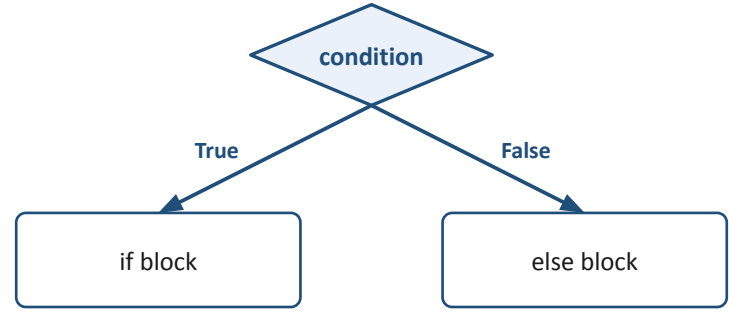


False: condition skips straight past, block never runs

```
age = 20
if age >= 18:
    print("You are an adult")
```

if...else Statement

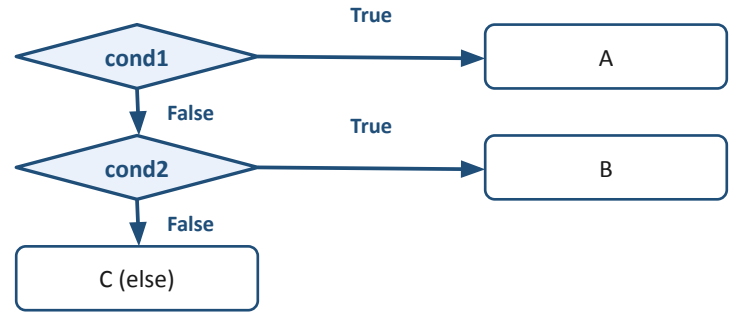
Runs one block if the condition is True, another if it's False.




```
age = 15
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

if...elif...else Statement

Checks multiple conditions in order, top to bottom.




```
marks = 72
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F")
```

Comparison & Boolean Operators

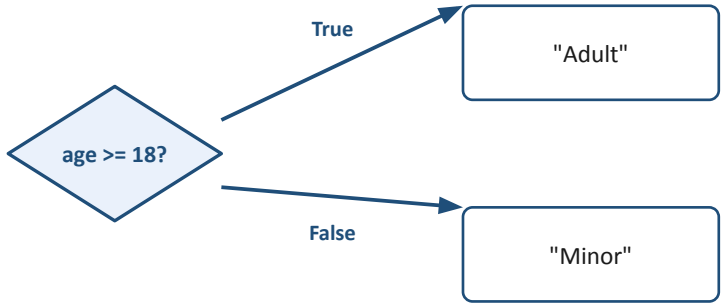
Comparison: == != > < >= <= Boolean: and, or, not

A	B	A and B
True	True	True
True	False	False
False	False	False

```
age = 25
has_id = True
if age >= 18 and has_id:
    print("Entry allowed")
```

Ternary (Conditional) Expressions

A one-line shorthand for if...else. Common in interviews.




```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

EXAMPLE 1

Practice: Check Even or Odd

Type this code yourself and run it.

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(f"{num} is even")
else:
    print(f"{num} is odd")
```

EXAMPLE 2

Practice: Largest of Three Numbers

Type this code yourself and run it.

```
a, b, c = 12, 45, 7
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print("Largest:", largest)
```

EXAMPLE 3

Practice: Simple Grading System

Type this code yourself and run it.

```
marks = int(input("Enter marks: "))
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "F"
print("Grade:", grade)
```

EXAMPLE 4, INTERVIEW FAVORITE

Practice: Leap Year Checker

Type this code yourself and run it.

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a leap year")
else:
    print(f"{year} is not a leap year")
```

QUESTION 1

Think & Practice

Write a program that checks whether a given number is positive, negative, or zero.

ANSWER

Answer 1

```
num = int(input("Enter a number: "))
if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

QUESTION 2, INTERVIEW STYLE

Think & Practice (Harder)

A triangle has three sides a, b, and c.

Write a program that checks whether the three sides can form a valid triangle.

Hint: the sum of any two sides must be greater than the third side.

ANSWER

Answer 2

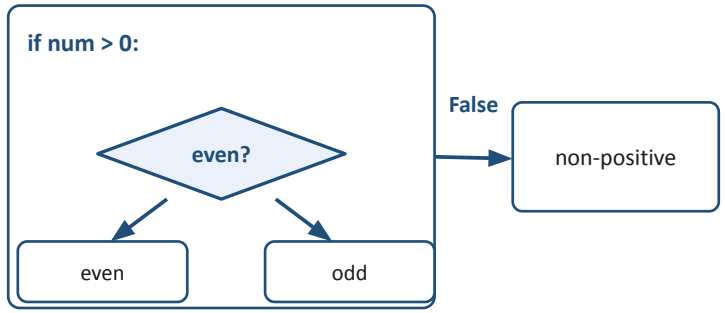
```
a, b, c = 7, 10, 5
if (a + b > c) and (a + c > b) and (b + c > a):
    print("Valid triangle")
else:
    print("Not a valid triangle")
```



Hour 2: Nested Conditionals, Truthiness & Bugs

Nested Conditionals

An if statement placed inside another if statement.




```
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
else:
    print("Non-positive number")
```

What Is Truthiness?

- Every value in Python is either "truthy" or "falsy" when used in an if
- Falsy: 0, 0.0, "" (empty string), [] (empty list), {} (empty dict), None, False
- Truthy: pretty much everything else, non-zero numbers, non-empty strings/lists
- This lets you write if my_list: instead of if len(my_list) == 0:

Truthiness in Practice

Checking emptiness directly is more "Pythonic" than comparing lengths.

```
name = ""
scores = [85, 90]

if name:
    print("Name was provided")
else:
    print("Name is empty")

if scores:
    print("There are scores to show")
```

Common Conditional Bugs

- Using = instead of ==: Python raises a SyntaxError immediately, so this one is at least caught fast
- Comparing floats with ==: 0.1 + 0.2 == 0.3 is actually False, due to floating-point precision
- Stacking separate if statements when you meant elif, so more than one block can run
- Misreading and/or: "age > 18 or age < 5 and is_student" does not group the way it reads aloud

The Classic Float Trap

A famous "gotcha", worth seeing once so it never surprises you in an interview.

```
print(0.1 + 0.2)        # 0.30000000000000004
print(0.1 + 0.2 == 0.3)  # False!

# Safer: compare within a small tolerance
print(abs((0.1 + 0.2) - 0.3) < 1e-9)  # True
```

QUESTION 1

Think & Practice

Given a list, print True only if the list is non-empty and its first element is positive, using truthiness rather than len().

ANSWER

Answer 1

```
items = [5, 2, 9]
print(bool(items) and items[0] > 0)

items = []
print(bool(items) and items[0] > 0)   # short-circuits, no IndexError
```



15-Minute Break

Stretch, grab a coffee. We resume with Hour 3: Reading a Traceback.



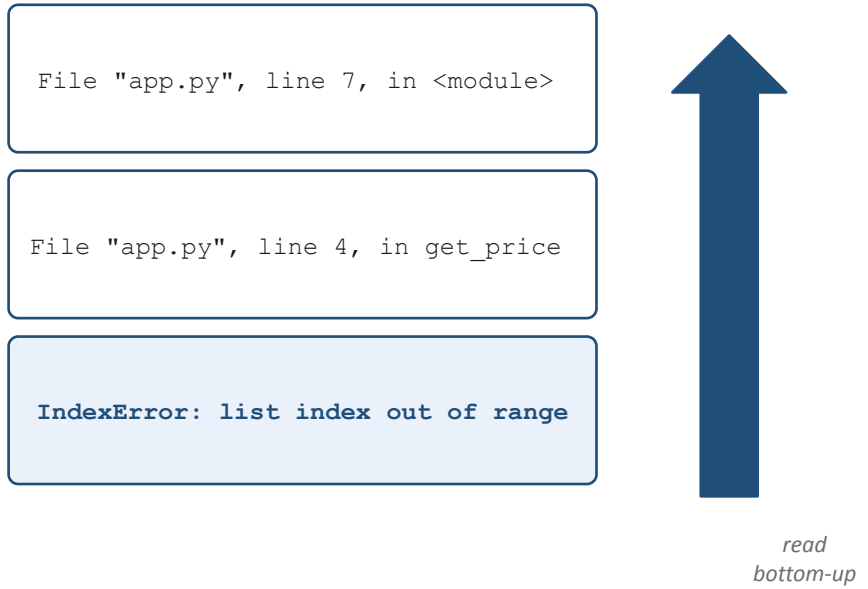
Hour 3: Reading a Traceback

Syntax Errors vs. Runtime Errors

- A syntax error means Python could not even understand your code, it never runs at all
- A runtime error means your code was valid Python, but something went wrong while it was executing
- Syntax errors are caught immediately when you try to run the file
- Runtime errors only show up when that specific line actually executes

How to Read a Traceback, Bottom-Up

- Start at the very last line, that's the error type and the message
- The line just above it shows the file and line number where it happened
- If there are several "frames," the bottom one is closest to the actual failure
- Read bottom to top, not top to bottom, most beginners read it backwards



NameError

Happens when you use a variable that was never defined.

```
print(total)

# Traceback (most recent call last):
#   File "app.py", line 1, in <module>
#     print(total)
# NameError: name 'total' is not defined
```

TypeError

Happens when an operation gets a value of the wrong type.

```
age = 20
print("Age: " + age)

# Traceback (most recent call last):
#   File "app.py", line 2, in <module>
#     print("Age: " + age)
# TypeError: can only concatenate str (not "int") to str
```

IndexError & KeyError

Happens when you access a position or key that doesn't exist.

```
scores = [90, 85, 77]
print(scores[5])
# IndexError: list index out of range

student = {"name": "Ali"}
print(student["age"])
# KeyError: 'age'
```

ZeroDivisionError

Happens when you divide by zero, always a runtime error, never a syntax error.

```
total = 100
count = 0
print(total / count)

# Traceback (most recent call last):
#   File "app.py", line 3, in <module>
#     print(total / count)
# ZeroDivisionError: division by zero
```

DEBUG THIS

Think & Practice

You run this code and get: IndexError: list index out of range

at File "app.py", line 3, in <module>

fruits = ["apple", "banana"]

print(fruits[0])

print(fruits[2])

Which line failed, and why?

ANSWER

Answer

```
# Line 3 failed: print(fruits[2])
# The list only has indexes 0 and 1 (two items),
# so index 2 does not exist.

fruits = ["apple", "banana"]
print(fruits[0])
print(fruits[1])   # fixed: last valid index is 1
```

Homework: Practice for Interviews

Try these on your own before next class:

1. Write a program that checks whether a number is a whole number of dozens (divisible by 12), without loops.

2. Write a program using truthiness to check whether a string is empty before processing it.

3. Deliberately write code that causes a TypeError, then fix it. Save both versions.

4. Deliberately write code that causes a KeyError, then fix it using .get() instead.

5. Explain, in your own words, the difference between a syntax error and a runtime error.

Key Takeaways

- if / elif / else make decisions based on conditions, checked top to bottom
- Empty values, 0, and None are all "falsy", you can check them directly in an if
- A traceback is read bottom-up, the last line names the error, the line above names the location
- Being able to read your own errors is a core skill, not a sign you did something wrong

Coming Up: Day 3

- Loops
- for loops and range(), iterating over strings and lists
- while loops, and loop control: break, continue, pass
- Problem set: classic interview warm-ups including swapping variables and FizzBuzz